



QwenPI VLA Training Optimization

NVIDIA DevTech

Results at a Glance

starVLA QwenPI: Qwen3.5-VL 4B + flow-matching action head, ZeRO-2 full fine-tune

2.6×

Faster training step

800 → 306 ms/step (bs 8)

209

Samples / node / s

up from 80 at hand-off

18.8%

**Model FLOPs
utilization**

up from 7.2% (bf16 peak)

309

Samples / node / s

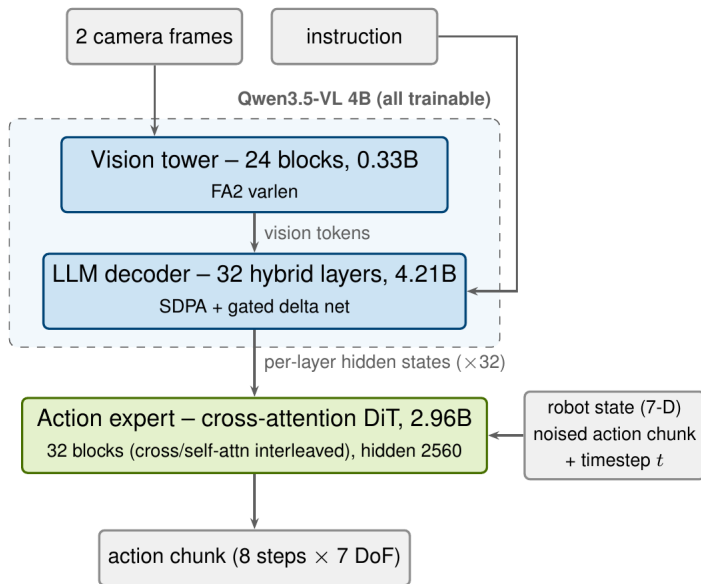
at bs 24 — 1.5× the 200
target

Method: every optimization was profiled with Nsight Systems and validated with a same-node A/B run plus loss-trajectory checks — no stacked guesses, measured deltas only.

Measurement: 8×H200 (NVLink); step time = inter-step period (includes the step-boundary parameter AllGather); confirmed by a clean 1000-step run at 303–313 ms sustained.

Model & Workload

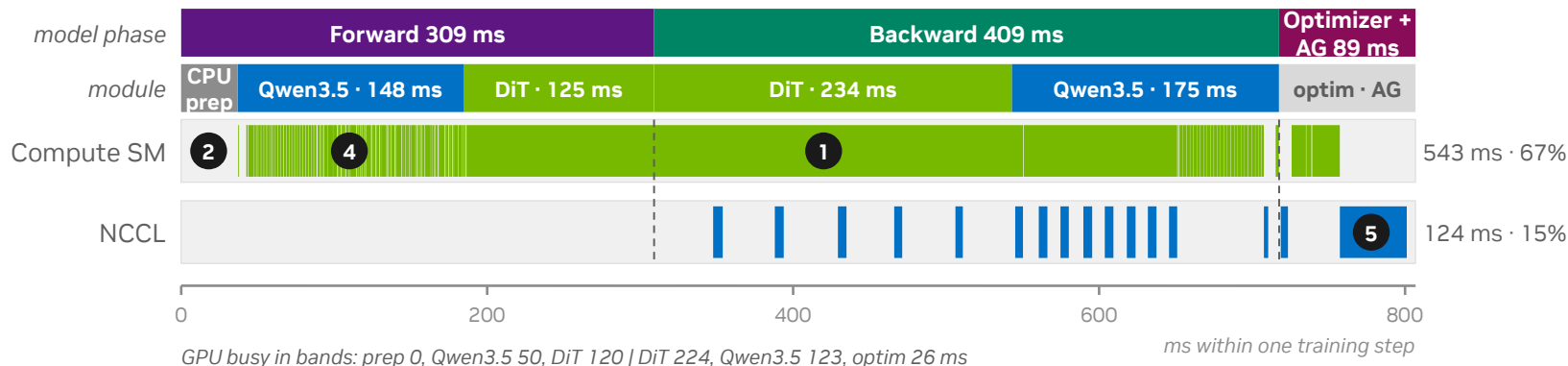
7.5B trainable parameters: Qwen3.5-VL 4B backbone + 3B flow-matching action expert



Training configuration	
Parameters	7.52B, all trainable (bf16)
Parallelism	DeepSpeed ZeRO-2, DP=8, 1 node
ZeRO-2 tuning	bucket 1.5e9, reduce_scatter off
Optimizer	AdamW, fp32 master (sharded)
Batch	8 / GPU → 64 global (up to 24/GPU)
Sequence	text fixed at 192 (left-pad)
Data	LIBERO manipulation, 2 views/sample
Hardware	8×H200 SXM, full NVLink
Software	torch 2.7.1 / NCCL 2.26.2 / HF 5.3 / FA2.8

Baseline Diagnosis: Where Did 800 ms Go?

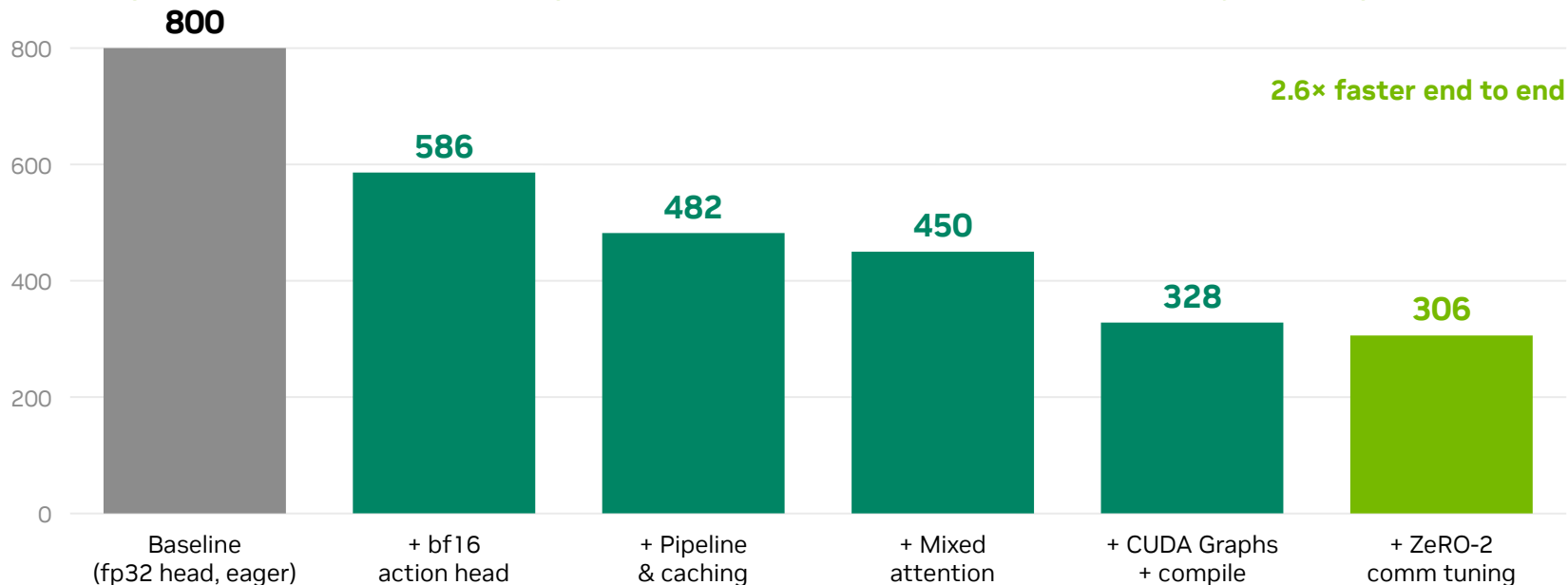
One baseline step on the GPU-execution clock (807 ms, bs 8/GPU) — five problems, five fixes



1 · fp32 action head	201 ms fp32 GEMMs vs 66 ms bf16 — 2.96B DiT outside autocast	bf16 DiT (−218 ms)
2 · CPU inside forward()	first ~40 ms of every step: compute idle, HF preprocessing on critical path	collate in DataLoader workers
3 · Per-step syncs	loss readback, DS timers, MRoPE position-ids, mm-merge check	sync-free logging + caches (2+3: −117)
4 · Launch-bound eager	13,868 launches @ 8 μs median; occupancy fwd 34% vs bwd 70%; idle 205 ms	CUDA Graphs + compile (−126 ms)
5 · Comm configuration	15 small AR buckets; 22 ms redundant flatten; exposed AllGather tail	bucket 1.5e9 + reduce_scatter off (−22)

Optimization Roadmap

Actual step time (ms) after each optimization lands — same node, steady-state p50



Bars are steady-state inter-step periods — they include the step-boundary AllGather drain that in-step timers miss. The torch 2.7.1 + NCCL 2.26.2 upgrade (~37 ms) is folded into the Pipeline bar. Final 306 ms confirmed by a clean 1000-step run.

Fix 1 · bf16 Action Head: -214 ms

800 → 586 ms. One config knob — and numerically a no-op for training

GPU busy by kernel class (baseline step, 543 ms)

201 ms

fp32 GEMM — all from the action head

126 ms

other (fla/GDN, embeddings, misc)

102 ms

elementwise / norm / reduce

66 ms

bf16 GEMM (Qwen3.5)

27 ms

copy / gather / cat

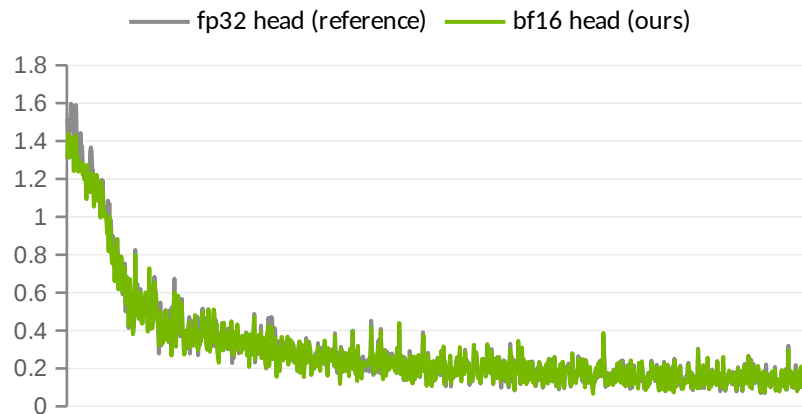
22 ms

attention kernels

37% of all GPU math is fp32 — all of it from the 2.96B DiT outside the bf16 autocast region.

Why was it fp32 at all? Inherited caution for diffusion numerics. Only the timestep embedding and the loss math need fp32 — we kept exactly those in fp32 and moved the transformer body to bf16 (dit_dtype knob, one-line rollback).

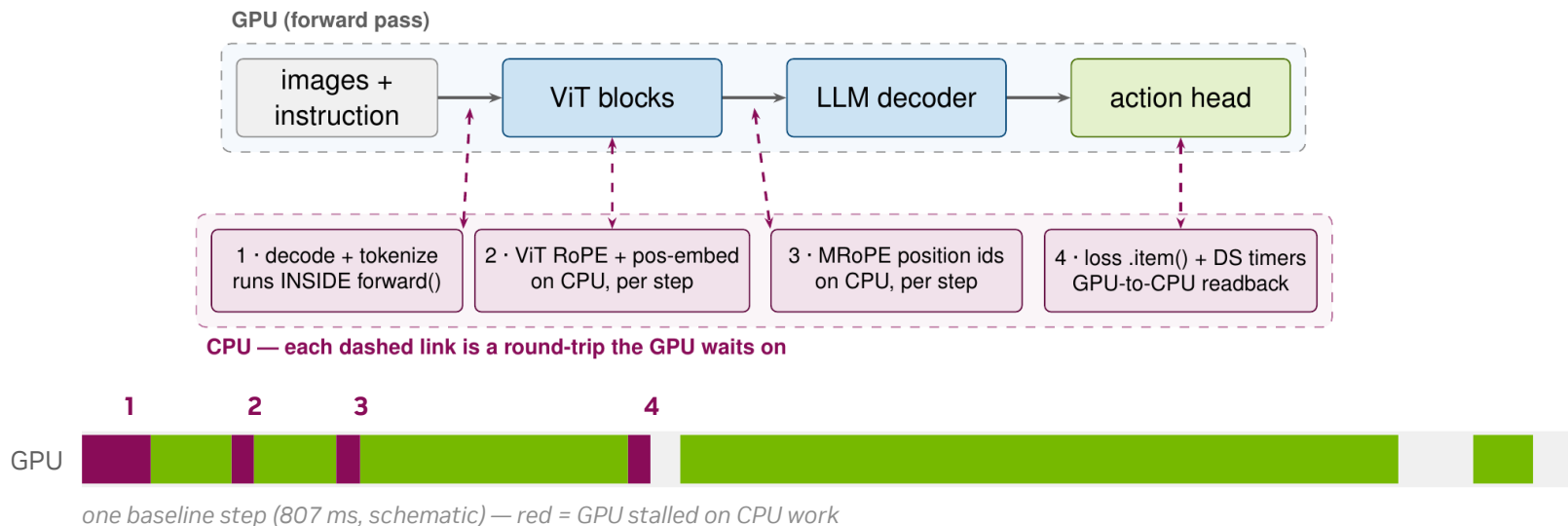
1,000 steps: the loss is unchanged



steps 1 → 1,000, only dit_dtype differs · mean $|\Delta| = 0.02$ (sampling noise) · final-100 mean 0.1473 vs 0.1473

Sync Points: Where the CPU Blocks the GPU

Four CPU round-trips hide inside every baseline training step

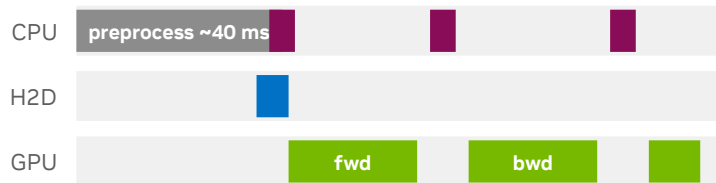


Measured: stall 1 $\approx$ 37 ms at every step start; stall 3 $\approx$ 13 ms; stall 4 drains the launch pipeline twice per step — together seeding the 205 ms/step of GPU idle. The next page removes all four.

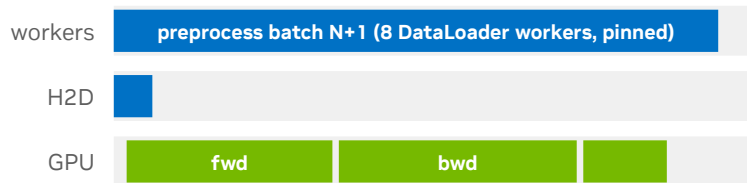
Removing the Sync Points: -67 ms

549 → 482 ms. Every CPU round-trip moves off the critical path

Before — everything serialized inside forward()



After — CPU work overlaps the previous step



red = per-step syncs (loss readback, DS timers, mm-merge check): each one drains the launch pipeline and strands the GPU

Preprocess in workers	HF processor (image patches + tokenize) runs in 8 workers, overlapped with the previous step	~40 ms off the critical path
Side-stream H2D	pinned host batch + dedicated copy stream, overlaps the ZeRO-2 optimizer tail	upload hidden
Sync-free logging	DeepSpeed timers unsynchronized; loss .item() only on logging steps	2 device syncs / step removed
MRoPE position-id cache	3D position ids precomputed on CPU, cached by image grid	~13 ms sync eliminated
+ Mixed attention	vision tower FA2 varlen (16 images / kernel); text stays SDPA — full FA2 measured slower at seq 192	-32 ms (482 → 450)

Why CUDA Graphs: Launch-Bound Forward

Anatomy of the Qwen3.5 forward — same math, very different wall clock

**Baseline
(eager)**



3,929 kernel launches · 19.1 ms of cudaLaunchKernel CPU time · occupancy 34%

**Optimized
(CUDA Graphs)**



6 graph launches + 736 eager · 9.4 ms launch CPU · occupancy 57%

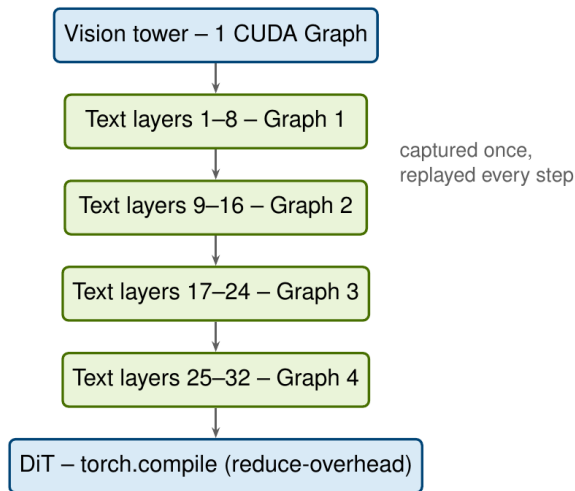
2.3× faster forward wall clock — with 81% fewer launches

- An eager op costs 30–60 μ s of Python dispatch + launch; the median kernel runs 8 μ s — the GPU finishes before the CPU can feed the next one
- Backward suffers far less (C++ autograd engine, occupancy 70%) — forward is where launch overhead bites
- A CUDA Graph replays an entire 8-layer group as ONE cudaGraphLaunch — per-op dispatch leaves the critical path

Busy also shrinks 50 → 37 ms (mixed attention + fusion); the split isolates the launch effect. Windows: baseline Qwen3.5 fwd vs merged-main VLM fwd, same launch-window accounting.

CUDA Graphs — and Why Four, Not One

450 → 328 ms (−122). Capture once, replay every step



Enablers: static shapes (pad-192) · vision tower = 1 graph · DiT = torch.compile · merge sync-free eager

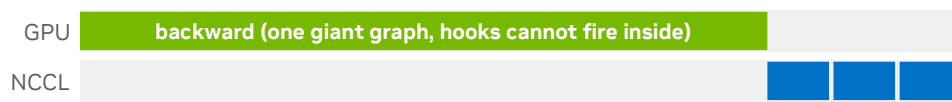
Why is the decoder FOUR graphs, not one?

DeepSpeed's compute-comm overlap must survive: AllReduce hooks fire only BETWEEN replays — never inside one.

4 graphs — AllReduce overlaps the group replays:



1 graph — every AllReduce waits for the whole backward:



same compute, same bytes — the red line is where the step ends

Kernel launches: 13.9k → 6.4k per step. CPU launch cost leaves the critical path; GPU idle collapses.

ZeRO-2 Comm Tuning: Two Knobs

328 → 306 ms. Two different problems: a redundant flatten, and a blocked backward

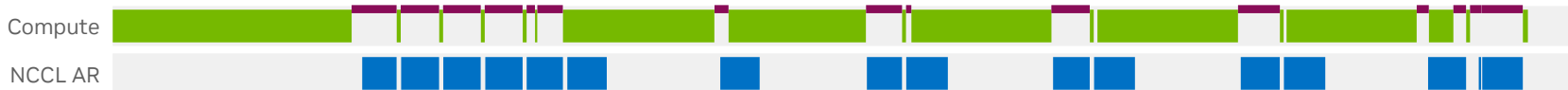
Knob 1 · `reduce_scatter` → `false` (−14 ms)

- Default path flattens every bucket (22 ms/step CatArrayBatchedCopy) — but grads already sit contiguous
- Plain AllReduce skips it. Cost: ~2× wire bytes — fine while comm overlaps and NVLink has headroom

Knob 2 · `reduce_bucket_size` → `1.5e9` (−13 ms)

- Grads double-buffer into TWO buckets; every boundary makes compute wait on the reduction stream
- Too small → backward stalls behind its own AllReduces (below). Too big → exposed tail (2e9 worse)

bucket 5e8 — backward 167 ms · 18 AllReduces · 15 stalls, 47 ms of blocked compute



bucket 1.5e9 — backward 154 ms · 8 AllReduces · 8 stalls, 27 ms

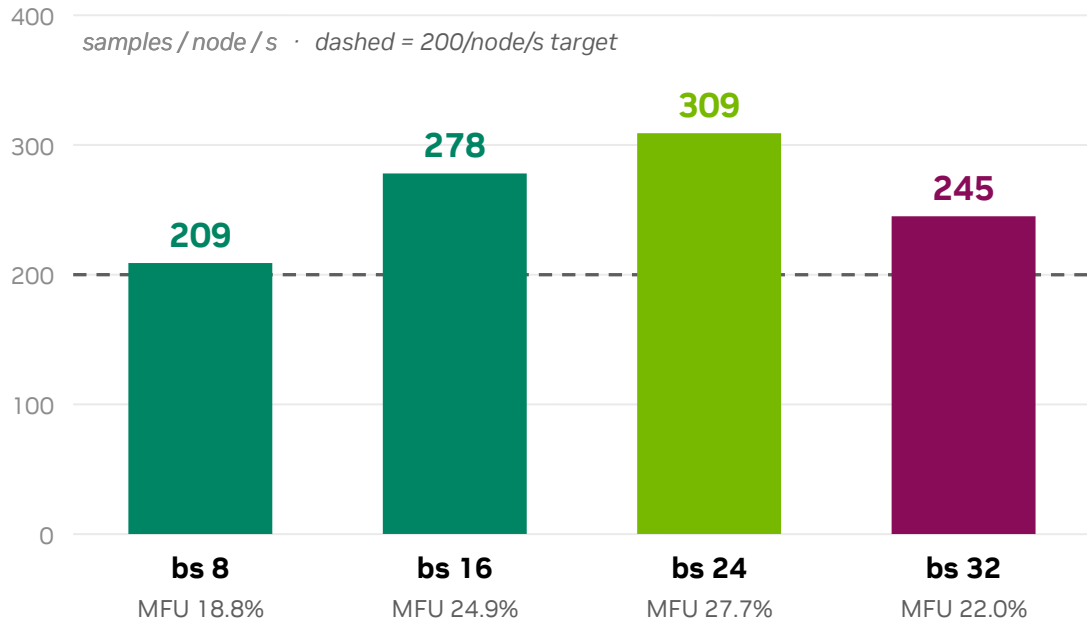


one backward pass, real kernels (node-level nsys, same node, same bytes) · red = compute stream stalled at a bucket rendezvous

No conflict — they compose: bucket size = how OFTEN compute and reduction rendezvous; `reduce_scatter` = what each rendezvous COSTS. `false` makes each AR bigger, strengthening the case for large buckets.

Batch Scaling: Sweet Spot at 24 per GPU

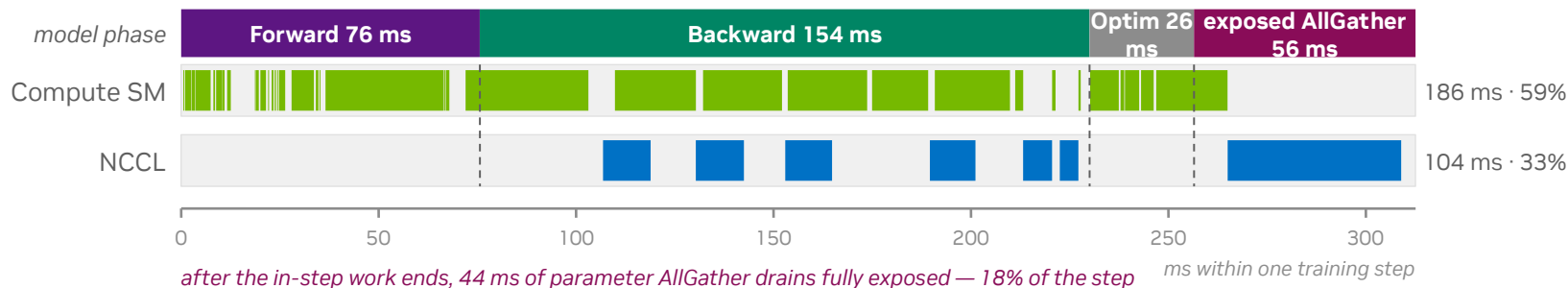
Same optimized config, only per-GPU batch changes — peak 309 samples/node/s (period basis)



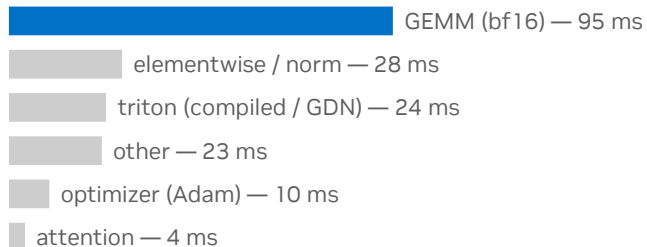
- Batching amortizes launch, comm and the AllGather drain: MFU climbs 18.8% → 27.7%
- bs 32 regresses hard (1043 ms/step): not OOM, not the dataloader — memory-pressure effects under investigation
- Larger global batch (192) changes optimization dynamics — needs a convergence run before adoption

What's Next

The optimized step, end to end — and where the next milliseconds live



GPU busy by kernel class (per step)



Flat profile — GEMMs already at the 622 TFLOPS power-wall ceiling.

Next levers (ranked by size)

Overlap the parameter AllGather — ~50 ms

ZeRO-2 cannot overlap it today: adopt FSDP (overlapped unshard) or extend ZeRO-2 with an async gather

fp8 for the GEMMs — path beyond the power wall

bf16 GEMMs are at the sustained 622 TFLOPS ceiling; fp8 is the only remaining math lever

Gradient compression / 1-bit Adam — ~20 ms

the in-step exposed AllReduce; bytes are the only lever (NVLink at line rate). Needs convergence sign-off